CapsuleOS: Recovery-Obligation Scheduling for Reversible Execution of Protected Edge Workloads

Sivaprakash S[1,*], Muthunagai S[1], Nivitha S[1], John Poly[1], Adith Krishna[1], Ivan Mathew[1]

[1] *Computer Science and Engineering, Vellore Institute of Technology, Vellore, India*

*Corresponding author. Email: sivaprakash.s@vit.ac.in

Abstract. Unfamiliar installers, scripts, and short-lived applications can create persistent host changes before an operator can determine whether execution is recoverable. Virtual machines offer a strong boundary but can be disproportionate for a short host-native workload on a constrained edge node. This paper presents CapsuleOS, a cooperating user-space prototype that treats recoverability as an admission resource. Before a registered persistent effect is exposed, CapsuleOS creates a recovery obligation, records inverse material, reserves recovery capacity, and evaluates edge constraints. It performs conflict-safe rollback and emits a hash-linked recovery certificate. In an end-to-end experiment, the prototype tracked four processes and four obligations, restored one modified file, removed three created files, and verified all obligations. A dynamic baseline trained from six verified runs accepted one normal holdout and identified a designed anomaly on five metrics. Transparent enforcement for arbitrary Windows applications remains future work.

Keywords: operating systems, edge computing, reversible execution, filesystem recovery, process scheduling, secure computing

1 Introduction

Edge computers and personal laptops routinely execute package installers, automation scripts, and third-party tools. Such workloads create descendants, alter host files, consume memory, or leave partially updated state after a failure. The practical question is whether a workload's host-visible effects can be reversed while preserving unrelated changes made by other software.

Monitoring and provenance systems explain activity after it occurs, but do not make a future write reversible. Filesystem journals and snapshots provide useful recovery primitives, but are usually not coupled to one process tree. Full virtual machines provide a guest boundary but need a guest OS, image storage, and lifecycle management, excessive for a short, explicitly scoped edge task. CapsuleOS targets a complementary middle ground: host-native execution with a recoverability contract for selected effects.

A protected operation is exposed only after the recovery information and capacity needed to reverse it have been prepared. The implementation represents this preparation as a recovery obligation recording a capsule identifier, resource identity, inverse material, state, and dependency information. The current prototype uses a cooperating user-space path: workloads request obligation preparation before exposing registered file effects. This paper therefore does not claim transparent interception of arbitrary binaries.

The contributions are: (1) a recovery-obligation lifecycle connecting protected file effects, capacity reservation, rollback, conflict handling, and a recovery certificate; (2) a process-tree-oriented recovery contract; (3) an edge-aware admission policy using recovery reserve, storage, memory, battery, flash-write budget, and rollback-time constraints; and (4) reproducible prototype evidence including end-to-end rollback and an explainable dynamic baseline.

2 Related Work and Research Gap

2.1 Provenance, recovery, and containment

HADES constructs whole-network provenance for Active Directory attack investigation, linking security events across machines [1]. Its purpose is attribution after activity is observed, whereas CapsuleOS uses operation records as prospective recovery material before a registered effect is exposed. APEX modifies Ext4 allocation to improve post-deletion recoverability on resource-constrained edge devices [2]; it shows recoverability and edge-storage overhead must be considered together, but is not a process-tree controller making inverse material and capacity an admission condition for individual host effects. Reversible-computing literature establishes the importance of causal reversal [3], but provides no implementation strategy for selected persistent effects on a commodity Windows host.

2.2 Snapshots, journaling, and transactional mechanisms

Three families of existing mechanisms are closest to CapsuleOS, and each differs in scope, trigger, and conflict semantics. *Volume or filesystem snapshots* (e.g., Volume Shadow Copy, Btrfs, ZFS) capture state at a point in time for a whole volume or subvolume. They are not scoped to one process tree, and restoring one typically reverts all changes made since the snapshot, including unrelated changes by other software; they also do not make snapshot capacity a precondition for each individual write. *Filesystem journals* (e.g., Ext4, NTFS) protect metadata or data consistency across crashes. They preserve filesystem integrity, not the ability to undo a completed, consistent application-level change. *Application-level transactions* (databases, package-manager rollback, or transactional filesystem APIs) require the application to be written for them; arbitrary installers and scripts are not, and the transactional NTFS API is deprecated by Microsoft.

CapsuleOS differs on four points: (i) the recovery unit is a capsule, i.e., a process tree, rather than a volume; (ii) inverse material and journal capacity are secured per registered effect *before* the effect is exposed; (iii) rollback is selective and, if the host version has changed independently, records a conflict instead of overwriting the other change; and (iv) the outcome is a hash-linked certificate. CapsuleOS is complementary to snapshots and journals and does not replace them (Table 1).

Table 1. Qualitative comparison with adjacent approaches.

Approach	Relationship to CapsuleOS
Monitoring	Offers observation but not a recovery precondition.
Volume snapshot	Restores a whole volume or subvolume to a point in time; not scoped to a process tree, and a restore also reverts unrelated changes.
Filesystem journal	Protects filesystem consistency across crashes; does not undo a completed, consistent change.
Application-level transaction	Atomic and reversible, but only for applications written to use it; arbitrary installers and scripts are not.
Container	Provides process isolation and limits but not selective host-effect rollback by default.
Full VM	Provides stronger guest isolation; CapsuleOS does not replace it.
CapsuleOS prototype	Provides a cooperating, task-scoped reversible workflow for registered files.

2.3 Checkpointing and fault-tolerant scheduling

Oikonomou et al. combine uncertainty-aware workflow scheduling with proactive checkpoint placement [4]. Hagras and El-Sayed use criticality and VM qualification for fault-tolerant, deadline-aware task assignment [5]. Both lines of work protect execution progress or service completion. CapsuleOS instead protects the host-visible persistent state affected by a task, preserving an inverse before a registered operation becomes visible and retaining conflict-safe rollback semantics if the host changes independently.

2.4 Security and adaptive edge scheduling

Escaleira et al. identify fragmented security controls across runtime, network, function, orchestration, and data layers in serverless systems [6]. Al Shamaa and Shi survey security-aware scheduling across cloud, fog, and edge settings [7]. Sefati et al. combine LSTM forecasting with multi-objective optimization for multi-cloud scheduling [8]. Ghorbian et al. use energy state to select serverless IoT-edge schedulers [9]. Yu et al., Wu et al., and Cao et al. apply reinforcement learning or reliability-aware optimization to cloud-edge scheduling [10–12].

These studies establish that learning, energy, reliability, and latency are mature scheduling inputs, but not that a process should be admitted only when an operation-specific inverse and sufficient recovery capacity have been secured. CapsuleOS uses dynamic behaviour analysis as a secondary operational signal, not as the core safety decision. Table 2 summarises the closest prior work.

Table 2. Comparison with relevant prior work.

Work	Contribution	Difference from CapsuleOS
HADES [1]	Whole-network provenance investigation	Investigates activity after it occurs; CapsuleOS prepares recovery for registered local effects.
APEX [2]	Edge-aware filesystem allocation and recovery	No process-tree recovery contract or per-effect admission.
Workflow checkpointing [4]	Uncertainty-aware checkpoint placement	Protects execution progress rather than host resource state.
Security scheduling [7]	Security-aware cloud/fog/edge scheduling survey	Does not use recoverability as a hard admission condition.
Energy-aware IoT edge [9]	Energy-state scheduling	Energy is the primary objective; CapsuleOS uses it as one recovery constraint.

3 CapsuleOS Design

3.1 System model

CapsuleOS targets accidental, faulty, or insufficiently trusted short-lived workloads whose selected filesystem effects can be registered in advance. The host owner supplies a recovery contract containing protected paths, a journal limit, process limits, and edge-capacity thresholds. The model assumes the CapsuleOS control plane and journal store are trusted and does not claim rollback of network, physical-device, or unregistered registry effects.

3.2 Recovery obligations

Fig. 1 shows the controller architecture and Fig. 2 the admission workflow for a cooperating protected operation.

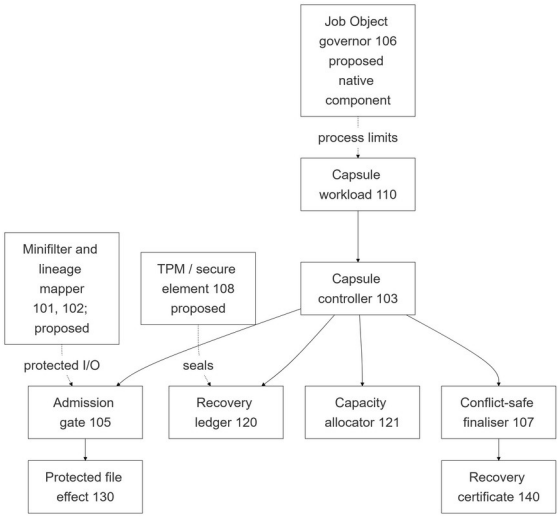


Fig. 1. CapsuleOS controller architecture. The diagram includes the evaluated user-space controller and the intended native enforcement blocks.

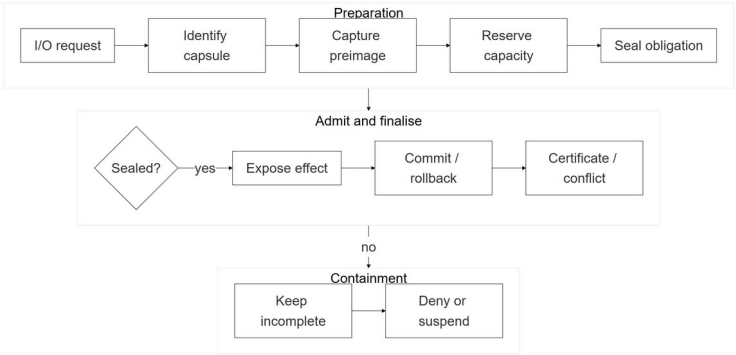


Fig. 2. Admission workflow for a cooperating protected operation.

For a registered file operation, CapsuleOS forms an obligation containing a capsule identifier, resource identity, original-state metadata (the pre-operation version), an inverse action, dependency data, a recovery-storage reservation, and a lifecycle state. The obligation states are `preparing`, `sealed`, `exposed`, `verified`, and three non-verified outcomes, `unrecoverable`, `expired`, and `conflicted`; Fig. 3 shows the transitions. A sealed obligation has its inverse action secured (a preimage backup for a modification, or a deletion marker for a created file) and any recovery storage it needs reserved; an exposed obligation is one whose cooperating operation has been allowed to proceed. An obligation becomes `unrecoverable` when the backup or reservation cannot be secured during preparation, and `expired` when a sealed admission record lapses before use; in neither case is the effect exposed. Commit and rollback are finalisation actions: both end in `verified` when the resource matches the expected version, and ledger tampering is detected at the ledger level (Sect. 3.4) rather than as an obligation state.

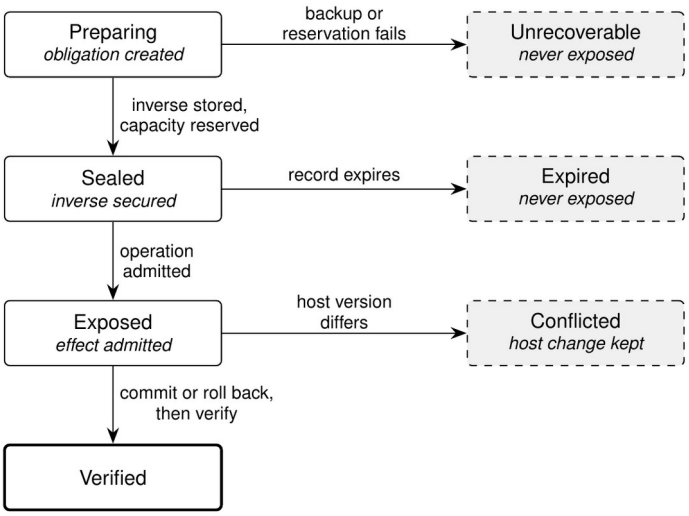


Fig. 3. Recovery-obligation lifecycle. Dashed grey states are non-verified outcomes: unrecoverable and expired obligations never expose the effect, and conflicted obligations are reported as unresolved in the certificate.

The recovery invariant is satisfied when every exposed registered effect has a sealed obligation and the capsule has no unresolved conflict; otherwise the prototype can deny a cooperating operation or suspend the workload. The intended native path uses Windows Job Objects and a Filter Manager minifilter, but this is outside the present evaluation (Fig. 4).

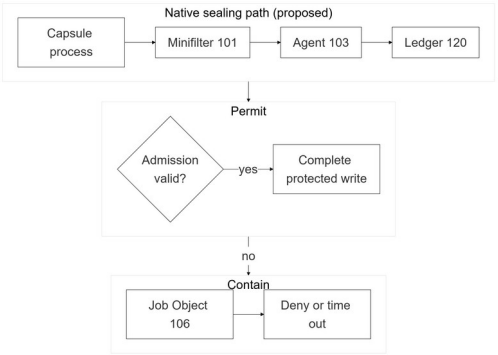


Fig. 4. Intended native enforcement sequence. This is an architecture figure, not an evaluated minifilter result.

Example obligation record. The records below are taken from the recovery ledger of the integration run of Table 3 (capsule 2f68619cb612) and show the modified file at three points of its life. Each ledger line wraps the obligation payload with a sequence number, event name, timestamp, and the hash link (`previous_hash`, `record_hash`). Digests are abbreviated, and fields marked ... are omitted for space.

```
(a) Before exposure: preparing
{ "sequence": 2, "event": "obligation_preparing",
  "payload": {
    "obligation_id": "62fb4a23679942cf9f4ee14b44e610e0",
    "capsule_id": "2f68619cb612",
    "resource_type": "file",
    "resource_id": "J:\\...\\workspace\\protected_note.txt",
    "operation": "write", "state": "preparing",
    "dependencies": [],
    "inverse_action": "restore_preimage",
    "backup_path": null,
    "reserved_bytes": 26,
    "pre_version": { "exists": true, "size": 26,
                     "digest": "77b978d2...c478" },
    "post_version": {},
    "admission_decision": "edge_admitted",
    "estimated_rollback_ms": 3,
    "capacity_snapshot": { ... },
    "sealed_at": null, "exposed_at": null, "verified_at": null },
  "previous_hash": "0000...0000",
  "record_hash": "e81777d7...2469" }


(b) After sealing, then exposure: sealed, then exposed
{ "sequence": 3, "event": "obligation_sealed",
  "payload": { ..., "state": "sealed",
    "backup_path": "...\\00000001_62fb4a23...bin",
    "sealed_at": "2026-07-28T07:32:18.212182+00:00" },
  "previous_hash": "e81777d7...2469",
  "record_hash": "d2d040f0...9b30" }
{ "sequence": 4, "event": "obligation_exposed",
  "payload": { ..., "state": "exposed",
    "post_version": { "exists": true, "size": 72,
                      "digest": "6d359549...d73c" },
    "exposed_at": "2026-07-28T07:32:20.643675+00:00" },
  "previous_hash": "d2d040f0...9b30",
  "record_hash": "54ac1a88...cd23" }


(c) After rollback: verified
{ "sequence": 20, "event": "obligation_verified",
  "payload": { ..., "state": "verified",
    "verified_at": "2026-07-28T07:32:22.861940+00:00" },
  "previous_hash": "e976b248...7cca",
  "record_hash": "daac7269...9e5a" }
```

In (a) the inverse action is chosen but `backup_path` is still null, so the recovery copy is not yet secured and the operation cannot proceed; `reserved_bytes` is 26, the size of the preimage. In (b) the preimage backup exists (`backup_path` is set) and the obligation is sealed; only then is the effect exposed, and `post_version` records the digest of the file the workload wrote, which rollback later compares against the host. Each record's `previous_hash` equals the preceding record's `record_hash`. In (c) the restored file again matches `pre_version` and the obligation is verified; this record's `record_hash` is the ledger head stored in the recovery certificate. If the host digest at rollback time differs from `post_version.digest`, a separate conflict record is created, the obligation becomes `conflicted`, and the file is not overwritten.

3.3 Finalisation, dynamic baseline, and edge policy

Finalisation freezes the capsule and checks resource versions. Rollback processes obligations in reverse dependency order: a created file is removed and a modified file is

restored from its preimage. When the current host version differs from that expected, the prototype records a conflict rather than overwriting the host change. Each finalisation creates a hash-linked certificate recording obligation counts, verification state, and unresolved effects.

The edge policy evaluates recovery reserve, storage, memory, battery, AC power, flash-write budget, and estimated rollback duration (Fig. 5). The dynamic baseline records process count, memory, file changes, write operations, obligations, journal use, runtime, and seal latency, trained only from verified, complete, conflict-free runs. For each metric it computes a rolling median, MAD, p95, and safety floor (Fig. 6).

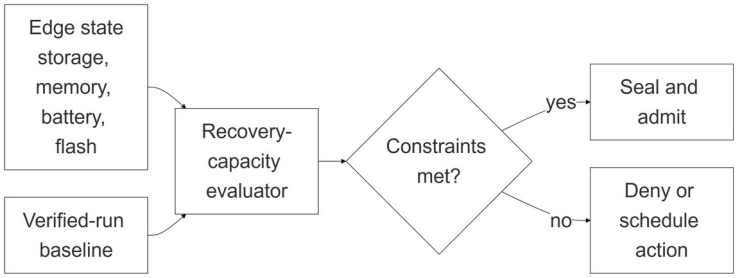


Fig. 5. Edge recovery-capacity admission policy.

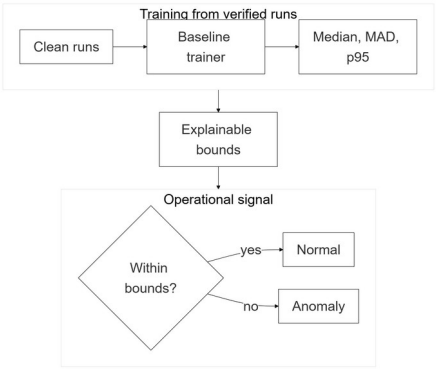


Fig. 6. Dynamic-baseline pipeline used to derive explainable operational bounds from verified runs.

On the intended edge hardware, the recovery-admission controller would combine power telemetry and a secure element with a kernel I/O boundary and a persistent recovery partition (Fig. 7); this boundary is not part of the evaluated prototype.

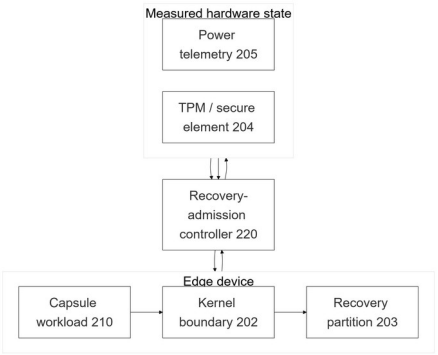


Fig. 7. Intended edge-hardware boundary with persistent recovery storage, kernel I/O enforcement, secure element, and power telemetry.

3.4 Failure and threat analysis

We analyse five scenarios against the system model of Sect. 3.1. The three baseline evidence runs contain no injected faults and no conflicts: the integration evidence reports zero pending obligations, zero unresolved conflicts, and no kept or skipped files at rollback. They do support two properties the analysis relies on. First, all four exposed effects had sealed obligations before rollback, with the invariant satisfied. Second, capacity accounting is exact: the recovery reserve fell from 16,764,902 B to 16,760,806 B, which is the 4,096 B the operation required. Four of the five scenarios are exercised by the fault-injection experiments of Sect. 5.4; bypassing the cooperating interface cannot be tested in a cooperating prototype and is analysed by design only.

Tampering with recovery records. The journal store is trusted and is expected to reside outside the registered paths. Each ledger record carries the hash of its predecessor, and the certificate's ledger head equals the last record's hash, so an edited record or a record removed from the middle of the ledger breaks the chain; verification then reports an integrity failure and the recovery invariant is violated. In the prototype this is a ledger-level condition rather than an obligation state. A tampered backup used for restore is detected at verification: the obligation becomes `unrecoverable` and the certificate reports recovery as incomplete (Sect. 5.4). Truncating the tail of the ledger or deleting the whole file is not detected, because the chain has no external anchor. A process running under the same account could also reach an unprotected journal directory, because journal and backup files are ordinary user-writable files; an external head anchor, restricted access-control lists, and the intended native path (minifilter-enforced journal access and TPM-backed anti-replay) are not yet built.

Termination during preparation. An effect is exposed only after its obligation is sealed. A process that dies while an obligation is `preparing` or `sealed` therefore leaves no exposed, unrecoverable effect on the cooperating path, and the host file is unchanged (confirmed by killing the process, Sect. 5.4). The abandoned obligation and its reservation are not reclaimed automatically, which is future work.

Journal unavailable or corrupted. If the journal cannot accept a new preimage or reservation, sealing fails (the obligation becomes `unrecoverable`) and the cooperating operation is denied or the workload suspended (the fail-closed branch of Fig. 3). Making the backup store unavailable confirmed this: the effect was not exposed, the host file was unchanged, and the invariant was reported as violated. Corruption of a record in the middle of the ledger is detected by the ledger integrity check, as described above. Journal replication is not part of the prototype.

Concurrent modification of a resource. Rollback compares the current host version against the expected post-write version. If another process, inside or outside the capsule, changed the resource, a conflict is recorded and the host change is preserved; both a second capsule writer and an external writer were detected in Sect. 5.4. The prototype has no mutual exclusion between writers, so detection happens after the fact and serialisation remains a property of the cooperating application.

Bypassing the cooperating interface. A workload that writes without requesting an obligation is not protected, and the prototype cannot detect it. This is the principal limitation of the user-space design and the motivation for the intended Filter Manager minifilter and Job Object path, which would deny unsealed writes to protected paths.

4 Implementation and Experimental Methodology

The prototype is implemented in Python 3.12 with FastAPI, Pydantic, and REST endpoints for the control plane and recovery contracts; `psutil` and Windows process APIs supply PCB-like process records and resource telemetry. Recovery uses file preimages, SHA-256 digests, append-only JSON records, and certificate verification. A small HTML/CSS/JavaScript dashboard displays the capsule lifecycle, obligations, changes, conflicts, and finalisation result. An automated suite of 28 unit tests (18 core tests and 10 reviewer-response tests), all of which pass, covers sealing before effect exposure, journal exhaustion and fail-closed behaviour, external-writer conflicts, certificate and hash-chain verification, ledger reload, file restoration and created-file removal, edge admission and denial, the baseline rules (warm-up, exclusion of unverified runs, and a 30-sample rolling window), and the process-tree, concurrency, failure-injection, and journal-failure scenarios of Sect. 5.4.

The evaluation asks: (RQ1) can the cooperating prototype maintain an obligation for each exposed registered effect; (RQ2) can it restore modified files and remove created files while producing a verifiable certificate; (RQ3) can the policy evaluate edge recovery capacity with low local decision latency; and (RQ4) can a verified-run baseline distinguish a normal holdout from a designed heavy workload?

Evidence files are `data/edge_integration_evidence.json`, `data/edge_admission_evidence.json`, `data/dynamic_baseline_evidence.json`, and `data/additional_test_results.json`. The integration workload performs a controlled modification and file creations in registered paths, with a 16 MiB recovery reserve and a 1.5 s rollback deadline. The edge benchmark evaluates five preimage sizes with 100 evaluations per scenario under a policy with a 32 MiB recovery reserve, at least 256 MiB free storage, 128 MiB free memory and 10% battery, a 5 s rollback limit, a 256 MiB flash-write budget, and an assumed restore throughput of 25 MiB/s. The baseline experiment comprises six verified runs, one normal holdout, and one deliberately heavy workload. The additional experiments of Sect. 5.4 use disposable temporary data on the same Windows 11 host (CPython 3.12.14) and the existing prototype modules, repeated 10 times for small configurations, 5 times for 100 files and 3 times for 1,000 files. These are mechanism demonstrations, not a broad malware corpus or a VM-performance study.

5 Results and Discussion

5.1 End-to-end recovery

The integration run tracked four processes and created four obligations; all four exposed effects were covered and the invariant satisfied, with final state `rolled_back`. Rollback restored one modified file, deleted three capsule-created files, and the certificate verified all obligations (Table 3). This answers RQ1 and RQ2 for registered, cooperating file effects; interception of effects that bypass the cooperating path is not established.

Table 3. End-to-end recovery evidence.

Measure	Result
Managed processes	4
Recovery obligations	4
Covered / exposed effects	4 / 4
Pending obligations	0
Unresolved conflicts	0
Restored modified files	1
Deleted created files	3
Verified obligations	4 / 4
Certificate verification	Passed

5.2 Edge-capacity admission

All five configured preimage scenarios were admitted under the chosen policy, each evaluated 100 times; p95 local decision latency ranged from 0.0166 ms to 0.0482 ms (Table 4). The rollback times in Table 4 are the policy's own estimates, computed from the configured restore throughput of 25 MiB/s; they grow with preimage size from 2 ms (0 B) to 322 ms (8 MiB), but they are not measured restores; measured rollback times are reported in Sect. 5.4.

These figures measure policy calculation only, not filesystem write latency, minifilter overhead, or VM comparison. RQ3 is answered for the local decision function.

Table 4. Edge-policy evidence. Estimated rollback time is computed by the policy, not measured.

Preimage	Recovery bytes	Estimated rollback time	p95 decision
0 B	4,096 B	2 ms	0.0355 ms
4 KiB	8,192 B	3 ms	0.0267 ms
64 KiB	69,632 B	5 ms	0.0166 ms
1 MiB	1,052,672 B	42 ms	0.0304 ms
8 MiB	8,392,704 B	322 ms	0.0482 ms

5.3 Dynamic baseline

The baseline trained from six verified, complete, conflict-free runs; the normal holdout remained inside all learned bounds, while the designed heavy workload exceeded five:

memory, process count, file changes, obligations, and runtime (Table 5). RQ4 is answered for these two cases, though the sample is insufficient for accuracy, precision, recall, or false-positive-rate claims. The heavy workload remained recoverable, showing that unusual behaviour and recoverability are separate properties.

Table 5. Dynamic-baseline evidence.

Metric	Holdout	Heavy	Upper bound
Peak memory	57,196,544 B	250,523,648 B	65,402,880 B
Process count	3	8	4
File changes	2	8	3
Obligations	2	8	3
Runtime	2,064.469 ms	3,097.787 ms	2,296.719 ms

5.4 Scaling and failure-injection experiments

To test whether obligation handling stays predictable as workload complexity grows, we ran additional experiments on the same host and prototype, using disposable temporary data. Table 6 varies the number and size of files, and Table 7 summarises process-tree depth, concurrent writers, failure injection, and journal failures.

File scaling. Every configuration and every repetition sealed, rolled back, and verified all files: the certificate reported recovery complete, no file was skipped, and no change remained after rollback. Sealing is a per-file cost that is flat for small files (about 10–15 ms from 0 B to 4 KiB) and grows sub-linearly with size (34 ms at 1 MiB, 68 ms at 8 MiB). Rollback time for the whole capsule grows with total bytes (302 ms for ten 8 MiB files) and with file count (6.5 s for 1,000 small files). Reserved bytes equal the preimage bytes exactly, one backup is written per file, and the ledger adds about 4 KB per file. The policy estimate of Table 4 is conservative for large files (302 ms measured against 3,220 ms estimated for ten 8 MiB files) but has no per-file term and underestimates 1,000 small files (6.5 s against 2.5 s); a file-count term is future work.

Table 6. File-scaling results. Seal latency is per file; rollback time is for the whole capsule. Every repetition was verified.

Files and sizes	Reps	Seal median / p95 (ms)	Rollback median / p95 (ms)	Policy est. (ms)	All verified
10 × 0 B	10	10.5 / 14.0	59.3 / 68.8	20	Yes
10 × 4 KiB	10	15.4 / 21.2	67.7 / 85.3	30	Yes
10 × 1 MiB	10	33.9 / 41.4	100.5 / 117.4	420	Yes
10 × 8 MiB	10	68.2 / 79.0	302.0 / 342.7	3,220	Yes
10 mixed (0 B–8 MiB)	10	15.2 / 63.4	112.7 / 121.4	743	Yes
100 mixed (0 B–1 MiB)	5	14.9 / 35.2	691.6 / 2,539.1	1,553	Yes
1,000 × 0–4 KiB	3	12.2 / 19.3	6,499.1 / 6,732.5	2,500	Yes

Process trees, concurrency, and failures. Parent–child process chains of depth 1, 4, and 8 were fully tracked, and the records were retained after termination for audit. Two

capsule writers on one file produced a detected lost update, and an external writer after exposure was detected and its bytes were preserved. Killing the process during preparation or after sealing exposed no effect, and after exposure the recovery material was intact and sufficient to restore the file. An interrupted rollback was resumable, and a backup store made unavailable caused the operation to fail closed.

Table 7. Process-tree, concurrency, failure-injection, and journal-failure results.

Experiment	Setting	Outcome
Process-tree depth	Depth 1, 4, 8	1, 4, and 8 records tracked; tracking latency 71, 624, and 775 ms
Concurrent writers	Two capsule writers, same file	2 obligations, 1 conflict: first writer's obligation conflicted, lost update detected
	External writer after exposure	Conflict detected; host bytes preserved (kept 1); invariant satisfied
Failure injection	Kill during preparing; kill after sealed	No exposed effect; host file unchanged
	Kill after exposed	Backup intact, matches preimage; manual restore verifies
	Interrupted rollback	Second pass completes: 4 of 4 files restored, 0 changes remain
Journal failures	Backup store unavailable	Obligation unrecoverable; effect not exposed; host unchanged; invariant violated (fail-closed)
	Ledger record removed from the middle	Integrity check fails (detected)
	Tampered restore backup	Detected at verification; recovery reported incomplete

These experiments also expose gaps. Recovery after a hard crash is not automatic: the ledger reloads, but the file-restore path must be rebuilt by hand, and abandoned reservations are not reclaimed. The hash chain detects edits and mid-chain removals but not truncation of the ledger tail or deletion of the whole file. There is no mutual exclusion between writers. Absolute latencies are host-dependent, with antivirus scanning active, and all results are from one host and cooperating workloads.

6 Threats to Validity and Conclusion

Our evidence is from one Windows 11 laptop running on battery power, and policy timing varies with load and storage conditions. The workload cooperates with obligation preparation. The results do not establish coverage for arbitrary installers, memory-mapped writes, registry changes, handles, device I/O, or network effects. The rollback times of Table 4 are policy estimates; measured rollback times come from one host (Sect. 5.4). The profile has only six normal runs and one designed anomaly, and is not a general benchmark. The failure and threat analysis of Sect. 3.4 combines design reasoning with the fault-injection results of Sect. 5.4, which cover cooperating workloads only. The intended Windows service, Job Object enforcement, Filter Manager minifilter, and TPM-backed anti-replay design have not been compiled or benchmarked.

CapsuleOS demonstrates a cooperating prototype for reversible execution of registered edge workloads, turning recovery information, journal capacity, and edge constraints into execution-admission inputs. In the recorded run, it covered four protected effects, restored one modified file, removed three created files, and verified the recovery certificate. Future work will implement native enforcement in a disposable environment, add automatic crash recovery, stale-reservation cleanup, and an external ledger anchor, add a per-file term to the rollback estimate, measure write-path overhead, add registry and memory-mapped-file coverage, and compare direct execution, CapsuleOS, Windows Sandbox, Hyper-V, and VirtualBox under identical workloads.

Acknowledgments

The authors acknowledge the support and facilities that enabled the prototype evaluation.

Disclosure of Interests

The authors declare no competing interests relevant to this work.

References

[1] Liu, Q., Bao, K., Hassan, W.U., Hagenmeyer, V.: HADES: Detecting and investigating active directory attacks via whole network provenance analytics. IEEE Transactions on Dependable and Secure Computing (2026)
[2] Tuli, S., Tuli, S., Jain, U., Buyya, R.: APEX: Adaptive Ext4 file system for enhanced data recoverability in edge devices. In: 2019 IEEE International Conference on Cloud Computing Technology and Science (2019)
[3] Ulidowski, I., Lanese, I., Mezzina, C.A., Tiezzi, F.: Reversible Computing: Extending Horizons of Computing. Springer (2020)
[4] Oikonomou, P., Kolomvatsos, K., Anagnostopoulos, C.: A proactive and uncertainty driven management mechanism for workflows of processing tasks. Computing 107, 112 (2025)
[5] Hagras, T., El-Sayed, G.A.: A fault-tolerant and load-balancing scheduler for independent tasks on cloud-based virtual machines. Cluster Computing 29, 61 (2026)
[6] Escaleira, P., Cunha, V.A., Barraca, J.P., Gomes, D., Aguiar, R.L.: A systematic review on security mechanisms for serverless computing. Cluster Computing 28, 465 (2025)
[7] Al Shamaa, S., Shi, W.: A systematic review of security-aware task scheduling in cloud, fog, and edge computing. Journal of Cloud Computing (2026)
[8] Sefati, S.S., Keymasi, M., Craciunescu, R., Maiduc, S., Bayram, M., Arasteh, B.: Adaptive resource scheduling in multi-cloud computing using recurrent neural forecasting and memory-based metaheuristic optimization. Journal of Grid Computing 23, 26 (2025)
[9] Ghorbian, M., Ghobaei-Arani, M., Esmaeili, L.: Autonomous scheduling mechanism based on energy awareness for improving resource allocation in serverless IoT edge. Scientific Reports 15, 24173 (2025)
[10] Yu, X., Mi, J., Tang, L., Long, L., Qin, X.: Dynamic multi objective task scheduling in cloud computing using reinforcement learning for energy and cost optimization. Scientific Reports 15, 45387 (2025)
[11] Wu, C., Ye, Q., Wang, Y., Zhang, D., Zhang, W., Jiang, X.: Research on cloud-edge-end distributed collaborative computing based on deep reinforcement learning. Scientific Reports 16, 2909 (2026)
[12] Cao, X., Chen, C., Li, S., Lv, C., Li, J., Wang, J.: Research on computing task scheduling method for distributed heterogeneous parallel systems. Scientific Reports 15, 8937 (2025)